Développer avec un agent de code : protocole des deux séances

Novembre 2026 — Supervision : Tanguy Vansnick

Résumé. En binômes, chacun sur son PC, les étudiants réalisent deux applications en deux séances de 2 h. La première se fait comme d'habitude, avec ou sans IA. La seconde se fait avec un agent de code, d'abord sans skill, puis avec des skills.

1 Objectifs

À l'issue des séances, l'étudiant sait partager un projet sur GitHub, utiliser un agent de code, vérifier ce qu'il produit et créer un skill simple.

2 Cadre

— **Groupes** : binômes fixes, un trinôme si le nombre est impair. Chacun travaille sur son PC, dans un seul dépôt par projet.
— **Technologies** : Python, SQLite, pytest, Git, GitHub, et au choix Flask ou Streamlit. L'organisation du code et l'interface sont libres. La base est fournie avec des exemples et reste modifiable.
— **Agents** : OpenCode (nombreux modèles gratuits, pas forcément européens) ou Mistral Vibe (modèle français, crédits gratuits).
— **But** : une application qui fonctionne, avec les fonctionnalités demandées.
— **À rendre** : le dépôt GitHub du binôme, avec son historique Git, poussé à la fin de la séance, et un README à jour si la façon de lancer l'application change.

3 Matériel

— **Projet 1, Partage de frais** : `github.com/umons-ig/codelab-sans-agent`.
— **Projet 2, Réservation de salles** : `github.com/umons-ig/codelab-agent`.
— **Skills** : `github.com/umons-ig/codelab-skills`.

4 Séance 1 — Partage de frais, comme d'habitude

4.1 Déroulé

Tableau 1. Déroulé de la séance 1.

Min	Étape
10	Présentation du projet et de GitHub
10	Création du dépôt (annexe A) et installation
95	Réalisation, comme d'habitude (annexe B pour travailler à deux)
5	Clôture

4.2 Ce que l'application doit faire

Un groupe d'amis part une semaine en vacances. Chacun paie des choses pour les autres, les tickets s'accumulent, et à la fin plus personne ne sait qui doit combien à qui. L'application note les dépenses et fait les comptes à tout moment.

Tableau 2. Demandes de l'application de la séance 1.

Données	Groupe : nom, dates du séjour, membres. Dépense : libellé, montant, date, payeur, participants.
Fonctionnalités	Créer un groupe et ses membres. Ajouter ou supprimer une dépense. Afficher le solde de chaque membre.
Règles	Une dépense a un montant positif et au moins un participant. Le payeur et les participants appartiennent au groupe. La date d'une dépense est comprise dans le séjour, dates de début et de fin incluses. Une dépense est partagée à parts égales entre ses participants. Le centime restant va au premier participant ajouté. Les soldes sont exacts au centime près et leur somme vaut toujours 0.
Si vous avez fini	Modifier une dépense, filtrer les dépenses par membre ou par période, exporter en CSV.

Tableau 3. Vérifications de la séance 1. Groupe : Alice, Bob et Chloé. Les lignes s'enchaînent.

Action	**Résultat attendu**
Alice paie 30 € pour Alice, Bob et Chloé	Alice +20,00 ; Bob −10,00 ; Chloé −10,00
Une dépense datée hors du séjour	Refusée, soldes inchangés
Une dépense de 0 €	Refusée, soldes inchangés
Sur un nouveau groupe, Alice paie 10 € pour les trois	Alice +6,66 ; Bob −3,33 ; Chloé −3,33

5 Séance 2 — Réservation de salles, avec un agent de code

5.1 Déroulé

Tableau 4. Déroulé de la séance 2.

Min	Étape
10	Présentation du projet
10	Démonstration de l'agent et de la revue de code
10	Création du dépôt (annexe A) et installation de l'agent
10	Lecture de AGENTS.md, déjà rempli
30	Réalisation sans skill
25	Réalisation avec les skills, puis création d'un skill pour la documentation
20	Demande de changement : chaque salle a ses propres horaires d'ouverture
5	Bilan

5.2 Ce que l'application doit faire

Dans un bâtiment universitaire, les salles se réservent par mail auprès du secrétariat, qui tient un tableur. Les demandes se croisent, les doubles réservations sont fréquentes et certaines salles sont réservées pour des groupes trop grands. L'application permet à chacun de réserver lui-même, avec des règles appliquées automatiquement.

Tableau 5. Demandes de l'application de la séance 2.

Données	Salle : nom, capacité, présence d'un projecteur. Réservation : salle, personne, date, heure de début et de fin, nombre de participants.
Fonctionnalités	Ajouter, modifier ou supprimer une salle. Réserver une salle au nom d'une personne. Annuler une réservation. Afficher le planning d'une salle sur une semaine.
Règles	Deux réservations d'une même salle ne se chevauchent pas. Finir à 12 h et commencer à 12 h est autorisé. Le nombre de participants ne dépasse pas la capacité de la salle. Une réservation a lieu entre 8 h et 20 h et dure de 30 min à 4 h. Supprimer une salle supprime ses réservations. Il n'y a pas d'authentification. La semaine va du lundi au dimanche.
Si vous avez fini	Chercher une salle libre selon le créneau et la capacité, afficher le taux d'occupation, exporter le planning.
Annoncé en séance	Chaque salle a ses propres horaires d'ouverture. Cette demande n'est pas dans l'énoncé remis aux étudiants.

Tableau 6. Vérifications de la séance 2. Salle S1 de capacité 20, déjà réservée le lundi de 10 h à 12 h.

Action	Résultat attendu
Lundi, 11 h à 13 h	Refusée : chevauchement
Lundi, 12 h à 13 h	Acceptée
Lundi, 14 h à 15 h, 25 participants	Refusée : capacité dépassée
Lundi, 7 h 30 à 9 h	Refusée : avant 8 h
Mardi, 14 h à 14 h 20	Refusée : moins de 30 min
Mardi, 8 h à 13 h	Refusée : plus de 4 h

5.3 Installer l'agent

Installer un des deux agents, puis taper sa commande dans le terminal, à la racine du dépôt.

Tableau 7. Installation et lancement des agents.

Agent	Installation	Lancement
OpenCode	`curl -fsSL https://opencode.ai/install | bash` ou `npm install -g opencode-ai`	`opencode`
Mistral Vibe	`curl -LsSf https://mistral.ai/vibe/install.sh | bash` ou `uv tool install mistral-vibe`	`vibe`

Sous Windows, `curl … | bash` ne fonctionne pas dans PowerShell. Pour OpenCode, utiliser `npm` (Node.js requis). Pour Mistral Vibe, installer uv, puis `uv tool install mistral-vibe`.

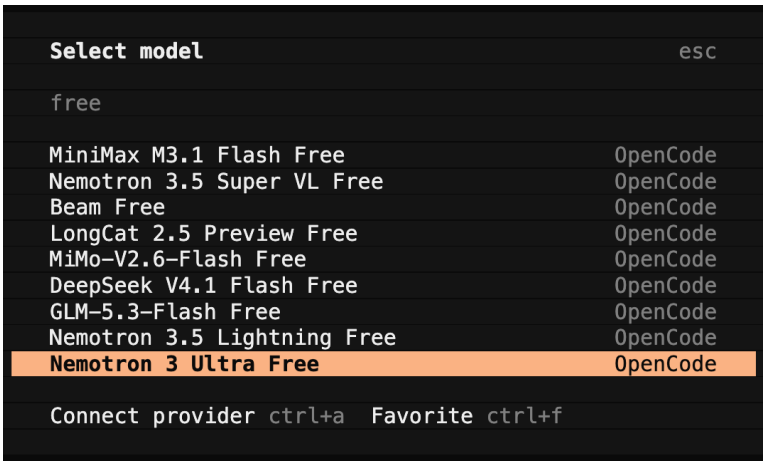


Figure 1. OpenCode : choisir un modèle gratuit avec `/models`, puis `free`.



Figure 2. Mistral Vibe après l'installation. Au premier lancement, il demande de se connecter avec un compte Mistral.

5.4 Relire chaque fonctionnalité

1. Faire un commit de la fonctionnalité.
2. Dans une nouvelle session, demander : `Fais une revue de code de git show, sans rien modifier`.
3. Lancer `pytest`, ouvrir la page et classer chaque remarque : juste, fausse ou inutile.
4. Faire corriger les remarques justes par l'agent.

5.5 Utiliser les skills

Les binômes clonent `github.com/umons-ig/codelab-skills` à côté de leur dépôt et copient les skills dans `.agents/skills/`. Un seul membre le fait et pousse, l'autre fait `git pull`. Les commandes sont dans le README. Avec Mistral Vibe, un skill s'appelle par `/nom-du-skill`. Avec OpenCode, on demande `Utilise le skill nom-du-skill`.

Tableau 8. Skills fournis.

Skill	Ce qu'il fait
`ajouter-fonctionnalite`	Ajoute une fonctionnalité avec ses tests et donne le vrai résultat de `pytest`.
`commit-push`	Lance `pytest`, écrit un message clair, puis pousse. S'arrête en cas de conflit.

5.6 Créer un skill pour la documentation

Chaque binôme crée ce skill pendant la réalisation avec les skills.

1. Avec Mistral Vibe, demander au créateur intégré : `/skill-creator Crée un skill expliquer-code : il lit git diff, résume en 5 lignes, dessine un diagramme de séquence et un flowchart`.
2. Avec OpenCode, demander à l'agent d'écrire le fichier : `.agents/skills/expliquer-code/SKILL.md`.
3. Relire le fichier créé et le raccourcir si besoin.
4. Ouvrir une nouvelle session et appeler le skill : `/expliquer-code`.

Annexe A Créer le dépôt

Pour les deux séances, depuis le dépôt modèle du projet.

1. Une seule fois par PC : `git config --global user.name "Votre nom"` et `git config --global user.email "votre@adresse.be"`. Si `git clone` refuse la connexion, installer GitHub CLI et lancer `gh auth login`.
2. Un membre clique sur *Use this template* sur le dépôt modèle et choisit *Private*.
3. Il ajoute son partenaire dans *Settings*, puis *Collaborators*. Le partenaire accepte l'invitation.
4. Chacun exécute `git clone`, puis suit le README.

Annexe B Travailler à deux

1. Chacun travaille sur son PC : `git pull` avant de travailler, `git push` après chaque commit.
2. Si `git push` est refusé, le partenaire a poussé entre-temps : `git pull --no-rebase --no-edit`, puis `git push`.
3. En cas de conflit : corriger les fichiers indiqués, puis `git add -A`, `git commit -m "fusion"` et `git push`.
4. `data.db` n'est pas dans Git : chacun a le sien. Si `base.sql` change, supprimer `data.db` et relancer.